

# Automatyzacje i integracje urządzeń w Home Assistant

Poradnik od podstaw do zaawansowanych przepisów: jak Home Assistant jest zbudowany, jakimi protokołami rozmawia z urządzeniami, jak pisać niezawodne automatyzacje — z wyjaśnieniami technicznymi dla ciekawych

wersja 1 — 6 lipca 2026 — dokument wewnętrzny projektu AMPERE\_POINT  
Uzupełnia: AMPERE\_POINT\_przewodnik\_HA\_instalacja\_integracja\_v2 (instalacja i konfiguracja od zera)  
oraz AMPERE\_POINT\_home\_assistant\_xtend\_tuya\_raport\_v1 (łańcuch Tuya w szczegółach).

## Spis treści

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>1 Wstęp — co jest w tym poradniku i jak go czytać</b>                      | <b>4</b>  |
| <b>2 Jak Home Assistant jest zbudowany</b>                                    | <b>4</b>  |
| 2.1 Encja, stan, zdarzenie — trzy pojęcia, na których stoi wszystko . . . . . | 4         |
| 2.2 Skąd HA pamięta historię: recorder i baza danych . . . . .                | 5         |
| 2.3 Integracje i wpisy konfiguracyjne . . . . .                               | 5         |
| 2.4 Warianty instalacji — przypomnienie w jednym akapicie . . . . .           | 6         |
| <b>3 Jak urządzenia trafiają do Home Assistant</b>                            | <b>6</b>  |
| 3.1 Odkrywanie: skąd HA wie, że coś jest w sieci . . . . .                    | 6         |
| 3.2 Cztery klasy łączności: lokalnie/chmurowo × push/poll . . . . .           | 6         |
| 3.3 Obie decyzje na jednym rysunku . . . . .                                  | 7         |
| <b>4 Protokoły — jak naprawdę rozmawia się z urządzeniami</b>                 | <b>7</b>  |
| 4.1 Wi-Fi i sieć domowa: HTTP, REST, WebSocket . . . . .                      | 7         |
| 4.2 MQTT — wspólny język automatyki . . . . .                                 | 8         |
| 4.3 Zigbee — radiowa siatka czujników . . . . .                               | 9         |
| 4.4 Z-Wave — siatka na osobnej częstotliwości . . . . .                       | 9         |
| 4.5 Thread i Matter — nowy porządek . . . . .                                 | 10        |
| 4.6 Bluetooth Low Energy . . . . .                                            | 10        |
| 4.7 Kamery: RTSP i ONVIF . . . . .                                            | 10        |
| 4.8 Modbus — język przemysłu (i falowników) . . . . .                         | 11        |
| 4.9 Ściągą porównawcza . . . . .                                              | 11        |
| <b>5 Ekosystemy w praktyce — czym i jak łączyć konkretne urządzenia</b>       | <b>11</b> |
| 5.1 Urządzenia Tuya (w tym ładowarki AMPERE POINT) . . . . .                  | 11        |

|           |                                                                               |           |
|-----------|-------------------------------------------------------------------------------|-----------|
| 5.2       | ESPHome — własne urządzenia w godzinę . . . . .                               | 12        |
| 5.3       | Shelly i podobne: Wi-Fi, ale lokalnie . . . . .                               | 12        |
| 5.4       | Telefon jako urządzenie: aplikacja Companion . . . . .                        | 12        |
| 5.5       | Co wybierać na przyszłość — krótki Kompas . . . . .                           | 13        |
| <b>6</b>  | <b>Anatomia automatyzacji</b>                                                 | <b>13</b> |
| 6.1       | Trzy człony i jeden tryb . . . . .                                            | 13        |
| 6.2       | Nowa i stara składnia YAML — jak czytać cudze przykłady . . . . .             | 13        |
| 6.3       | Przegląd wyzwalaczy — z krótkimi przykładami . . . . .                        | 14        |
| 6.4       | Warunki . . . . .                                                             | 15        |
| 6.5       | Akcje — pełny repertuar . . . . .                                             | 15        |
| 6.6       | Tryby wykonania . . . . .                                                     | 16        |
| <b>7</b>  | <b>Szablony Jinja2 — język, w którym HA myśli</b>                             | <b>16</b> |
| 7.1       | Absolutna podstawa: stany są tekstem . . . . .                                | 16        |
| 7.2       | Niezbędnik — funkcje i filtry, które załatwiają 95% potrzeb . . . . .         | 17        |
| <b>8</b>  | <b>Elementy pomocnicze: helpery, skrypty, sceny, blueprinty</b>               | <b>17</b> |
| 8.1       | Helpery — pamięć i pokręta Twoich reguł . . . . .                             | 17        |
| 8.2       | Skrypty i sceny . . . . .                                                     | 18        |
| 8.3       | Blueprinty — automatyzacje z formularzem . . . . .                            | 18        |
| <b>9</b>  | <b>Przepisy — kompletne automatyzacje z komentarzem</b>                       | <b>18</b> |
| 9.1       | Ładowanie w taniej taryfie — wersja dojrzała (helper schedule) . . . . .      | 19        |
| 9.2       | Powiadomienie o końcu ładowania — z danymi sesji . . . . .                    | 19        |
| 9.3       | Flagowy: ładowanie nadwyżką z fotowoltaiki z płynną regulacją prądu . . . . . | 20        |
| 9.4       | Alarm błędu z eskalacją — aż ktoś potwierdzi . . . . .                        | 21        |
| 9.5       | Powiadomienie z przyciskami (actionable) — pełny obieg . . . . .              | 22        |
| 9.6       | Watchdog niedostępności — system pilnuje sam siebie . . . . .                 | 23        |
| 9.7       | Raport dzienny o 21:00 . . . . .                                              | 23        |
| <b>10</b> | <b>Diagnostyka — gdy reguła nie robi tego, co miała</b>                       | <b>24</b> |
| 10.1      | Ślady (traces) — najpotężniejsze i najmniej znane narzędzie . . . . .         | 24        |
| 10.2      | Reszta niezbędnika . . . . .                                                  | 24        |
| 10.3      | Pułapki, które łapią każdego . . . . .                                        | 24        |
| <b>11</b> | <b>Pomiary energii — od encji do panelu Energia</b>                           | <b>25</b> |
| 11.1      | Co encja musi mieć, żeby panel ją przyjął . . . . .                           | 25        |
| 11.2      | Taryfy i okresy: utility_meter . . . . .                                      | 25        |
| <b>12</b> | <b>Niezawodność, kopie zapasowe, dostęp zdalny</b>                            | <b>25</b> |
| 12.1      | Kopie zapasowe . . . . .                                                      | 25        |

|                                              |           |
|----------------------------------------------|-----------|
| 12.2Aktualizacje bez dramatów . . . . .      | 25        |
| 12.3Dostęp spoza domu — trzy drogi . . . . . | 26        |
| <b>13Najczęstsze problemy — ściaga</b>       | <b>26</b> |
| <b>14Słowniczek pojęć i skrótów</b>          | <b>27</b> |

## 1. Wstęp — co jest w tym poradniku i jak go czytać

Home Assistant (dalej: HA) potrafi być jednocześnie prostym pilotem do włączania ładowarki i pełnoprawnym systemem sterowania budynkiem, który sam podejmuje decyzje. Różnica między jednym a drugim nie leży w sprzęcie, lecz w zrozumieniu trzech rzeczy: *jak HA jest zbudowany w środku, jak urządzenia się z nim komunikują oraz jak poprawnie zapisywać reguły automatyzacji*. Dokładnie w tej kolejności prowadzi ten poradnik.

Rozdziały 2-5 budują fundament: architektura systemu, protokoły komunikacji (od Wi-Fi i MQTT po Zigbee, Thread/Matter i Modbus) oraz przegląd ekosystemów urządzeń z praktycznymi wskazówkami. Rozdziały 6-8 to warsztat automatyzacji: anatomia reguły, język szablonów, elementy pomocnicze. Rozdział 9 zawiera siedem kompletnych, skomentowanych przepisów — od taniej taryfy po dynamiczne ładowanie nadwyżką z fotowoltaiki. Na końcu: diagnostyka (rozd. 10), pomiary energii (rozd. 11), niezawodność i dostęp zdalny (rozd. 12), tabela problemów i słowniczek.

Zasady zapisu są takie same jak w pozostałych dokumentach projektu: ścieżki menu zapisujemy jako **Ustawienia** → **Automatyzacje i sceny**, kod i identyfikatory czcionką maszynową (`sensor.moc`), a każdy skrót rozwijamy przy pierwszym użyciu. Ramki **Dla ciekawych** można pominąć bez szkody dla instrukcji — ale to w nich siedzi zrozumienie, *dlaczego* rzeczy działają.

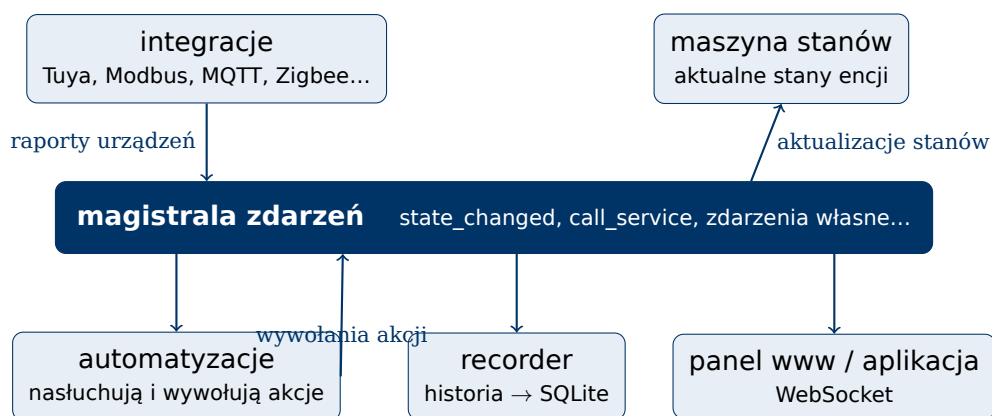
Wszystkie przykłady YAML (YAML Ain't Markup Language — tekstowy format konfiguracji, w którym wcięcia wyznaczają strukturę) zapisano w **nowej składni** automatyzacji, obowiązującej od wydania HA 2024.10; ramka w rozdziale 6 wyjaśnia, jak czytać starszą składnię, wciąż spotykaną w internecie.

## 2. Jak Home Assistant jest zbudowany

### 2.1. Encja, stan, zdarzenie — trzy pojęcia, na których stoi wszystko

**Encja** (entity) to najmniejsza jednostka, jaką HA widzi: jeden czujnik, jeden przełącznik, jedna wartość. Każda encja ma identyfikator złożony z domeny i nazwy (`sensor.ladowarka_moc`, `switch.ladowarka`), bieżący **stan** zapisany zawsze jako tekst (o tym ważnym szczególe więcej w rozdz. 7) oraz **atrybuty** — dodatkowe pola, np. jednostkę miary czy ikonę. Encje grupują się w **urządzenia** (device — teczka na encje z jednego fizycznego sprzętu), a urządzenia w **obszary** (pokoje, strefy budynku).

Sercem systemu jest **maszyna stanów** (state machine): tabela w pamięci, w której każda encja ma swój aktualny stan. Każda zmiana stanu generuje **zdarzenie** (event) `state_changed`, publikowane na wewnętrznej **magistrali zdarzeń** (event bus). Magistrala działa jak tablica ogłoszeń: kto chce, ten nasłuchuje. Automatyzacje to w istocie właśnie nasłuchiwanie — ich wyzwalacze są subskrypcjami zdarzeń.



**Dla ciekawych:** Cały HA jest programem w języku Python zbudowanym wokół jednej *pętli zdarzeń* (asyncio): zamiast wielu wątków czekających na urządzenia, jedna pętla obsługuje tysiące operacji naprzemiennie, przełączając się przy każdym oczekiwaniu na sieć. Dlatego HA na Raspberry Pi potrafi obsłużyć setki encji — większość czasu i tak czeka na odpowiedzi urządzeń. Konsekwencja praktyczna: pojedyncza źle napisana integracja, która blokuje pętlę, potrafi „przyciąć” cały system — stąd ostrożność przy dodatkach spoza oficjalnego zestawu.

## 2.2. Skąd HA pamięta historię: recorder i baza danych

Bieżące stany żyją w pamięci, ale zakładka **Historia** i wykresy wymagają zapisu na dysk. Robi to komponent **recorder**, który strumień zdarzeń zapisuje do bazy danych — domyślnie **SQLite**, czyli bazy w jednym pliku (`home-assistant_v2.db` w katalogu konfiguracji). Obok pełnej historii (domyślnie 10 dni) HA prowadzi **statystyki długoterminowe**: raz na godzinę zapisuje minimum, maksimum i średnią (dla liczników — przyrost) i trzyma je bezterminowo. To dzięki nim panel Energia pokazuje zeszły rok, choć szczegółowa historia dawno się przewinęła.

**Wskazówka:** Im mniej encji-śmieci, tym mniejsza i szybsza baza. Encje, których nie potrzebujesz, wyłączaj w **Ustawienia** → **Urządzenia i usługi** → **Encje**; głośne, a potrzebne tylko „na teraz” (np. siła sygnału) można wykluczyć z zapisu w konfiguracji recordera.

## 2.3. Integracje i wpisy konfiguracyjne

**Integracja** to moduł tłumaczący świat zewnętrzny na encje. Skonfigurowana integracja zostawia po sobie **wpis konfiguracyjny** (config entry) w ukrytym katalogu `.storage` — zapamiętane konto, adres, tokeny. Ta sama integracja może mieć wiele wpisów (dwa konta Tuya, trzy bramki). Z projektu znamy żelazną regułę: *pliki integracji bez wpisu konfiguracyjnego nie robią nic* — kreator (config flow) trzeba przejść do końca.

**Dla ciekawych:** Obok rejestru encji HA prowadzi rejestry urządzeń i obszarów (również w `.storage`). Dzięki nim zmiana nazwy encji nie psuje automatyzacji budowanych w edytorze graficznym — edytor zapisuje odwołania do identyfikatorów rejestrowych, nie do nazw. W YAML pisanym ręcznie odwołujemy się jednak do `entity_id` — dlatego po zmianie identyfikatora encji trzeba poprawić automatyzacje, które go używają.

## 2.4. Warianty instalacji — przypomnienie w jednym akapicie

Szczegóły instalacji opisuje przewodnik (dokument towarzyszący); tu tylko mapa: **HA OS** (np. na Raspberry Pi) to pełny zestaw z Supervisorem, sklepem dodatków i kopiami zapasowymi; **HA Container** (nasz wariant Docker na Windows) to sam rdzeń — lżejszy, ale bez sklepu dodatków. W obu przypadkach automatyzacje, integracje i wszystko, co opisuje ten poradnik, działa identycznie. Różnica praktyczna pojawi się tylko tam, gdzie wspominamy o *dodatkach* (add-ons) — w wariantcie kontenerowym ich odpowiedniki uruchamia się jako osobne kontenery Dockera.

## 3. Jak urządzenia trafiają do Home Assistant

### 3.1. Odkrywanie: skąd HA wie, że coś jest w sieci

Po starcie HA nasłuchuje kilku mechanizmów ogłaszania się urządzeń i pokazuje znaleziska w **Ustawienia** → **Urządzenia i usługi** jako karty „Wykryto”:

#### mDNS / zeroconf

urządzenia rozgłaszają w sieci lokalnej swoją nazwę i typ usługi (np. drukarka, ESPHome, Apple HomeKit) — to ten sam mechanizm, dzięki któremu działa adres `homeassistant.local`.

#### SSDP / UPnP

(Simple Service Discovery Protocol / Universal Plug and Play) — starszy mechanizm ogłoszeń używany m.in. przez telewizory, odtwarzacze i routery.

#### Nasłuch DHCP

HA obserwuje, jakie urządzenia pobierają adresy z routera; po adresie sprzętowym MAC (Media Access Control — unikalny identyfikator karty sieciowej) i nazwie potrafi rozpoznać znane rodziny sprzętu.

#### Bluetooth

jeżeli komputer z HA ma adapter Bluetooth (albo tzw. proxy — rozdz. 4.6), HA odbiera rozgłoszenia BLE czujników.

Odkrycie to tylko zaproszenie — integracja i tak przechodzi swój kreator (konto, kod PIN, potwierdzenie).

### 3.2. Cztery klasy łączności: lokalnie/chmurowo × push/poll

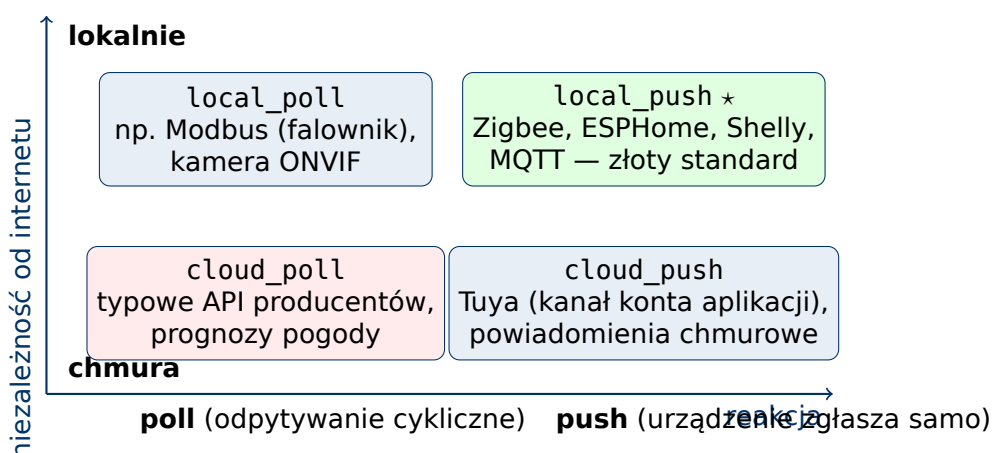
Każda integracja w oficjalnej dokumentacji ma zadeklarowaną tzw. klasę IoT (Internet of Things — internet rzeczy), złożoną z dwóch decyzji:

- **Lokalnie czy przez chmurę?** Lokalnie = HA rozmawia z urządzeniem bezpośrednio po sieci domowej (działa bez internetu). Chmurowo = pośredniczy serwer producenta (jak w Tuya — bez internetu encje stają się „niedostępne”).
- **Push czy poll?** Push = urządzenie samo zgłasza zmiany (natychmiastowe reakcje, mało ruchu). Poll = HA cyklicznie odpytuje (opóźnienie do długości cyklu; typowo 10-60 s).

Kombinacja `local_push` jest złotym standardem (Zigbee, ESPHome, Shelly po włączeniu zdarzeń); `cloud_poll` — najwolniejszym, ale często jedynym, co oferuje producent. Warto znać klasę swoich integracji, bo wyjaśnia ona zachowania: czujnik Zigbee reaguje w ułamku sekundy, a chmurowy odpytywany co minutę — właśnie co minutę.

**Dla ciekawych:** Skąd chmurowe integracje „push” znają zmiany bez odpytywania? Utrzymują stałe połączenie zwrotne — najczęściej WebSocket (kanał dwukierunkowy otwarty przez przeglądarkowy protokół HTTP) albo MQTT — którym serwer producenta wypycha zdarzenia. Tak działa m.in. kanał konta aplikacji w Tuya: integracja loguje się jak druga aplikacja mobilna i dostaje ten sam strumień powiadomień, który zasila telefon.

### 3.3. Obie decyzje na jednym rysunku



Gwiazdką oznaczono kwadrant, do którego warto migrować, gdy tylko urządzenie daje taką możliwość.

## 4. Protokoły — jak naprawdę rozmawia się z urządzeniami

Ten rozdział jest mapą świata urządzeń. Nie trzeba znać wszystkich protokołów, żeby korzystać z HA — ale kilka minut zrozumienia, czym różnią się drogi komunikacji, oszczędza później godziny zgadywania, czemu czujnik „muli”, a przełącznik nie reaguje.

### 4.1. Wi-Fi i sieć domowa: HTTP, REST, WebSocket

Najprostsza droga: urządzenie jest w tej samej sieci co HA i wystawia **API** (Application Programming Interface — „okienko podawcze” dla programów) po protokole **HTTP** — tym samym, którym przeglądarka pobiera strony. Styl takich API nazywa się **REST** (Representational State Transfer): proste żądania „pobierz stan” (GET) i „ustaw wartość” (POST) pod czytelnymi adresami. HA ma do tego gotowe integracje (`rest`, `rest_command`), a w drugą stronę — **webhooki**: unikalne

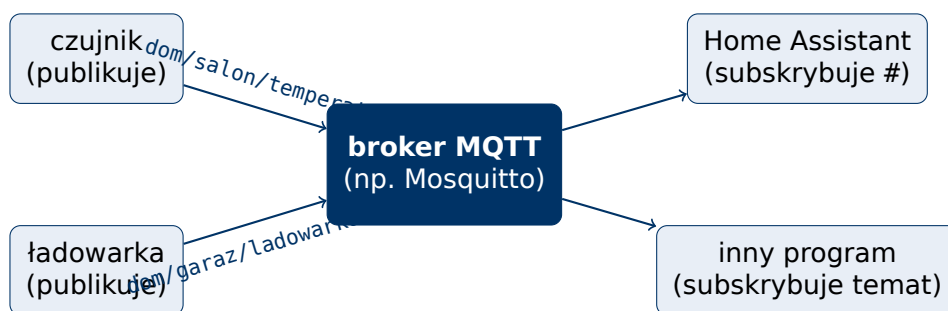
adresy URL, pod które dowolne urządzenie lub usługa może „zapukać” i tym samym wyzwolić automatyzację.

Tam, gdzie potrzebna jest ciągła dwukierunkowa rozmowa (panel www HA, ESPHome, Shelly nowszych generacji), używa się **WebSocket** — połączenia, które po nawiązaniu przez HTTP pozostaje otwarte i pozwala obu stronom wysyłać wiadomości w dowolnym momencie, bez odpytywania.

**Dla ciekawych:** Cały interfejs HA w przeglądarce to jedna aplikacja rozmawiająca z serwerem właśnie przez WebSocket. Gdy klikasz przełącznik, do HA leci wiadomość `call_service`, a z powrotem — zdarzenie `state_changed`. To dlatego dwa okna przeglądarki aktualizują się nawzajem „same z siebie”.

## 4.2. MQTT — wspólny język automatyki

**MQTT** (Message Queuing Telemetry Transport) to lekki protokół „publikuj-subskrybuj”, zaprojektowany dekady temu dla telemetrii rurociągów, a dziś będący lingua franca automatyki domowej. Zamiast rozmawiać każdy z każdym, wszyscy rozmawiają przez pośrednika — **brokera**:



Kluczowe pojęcia, które wystarczą do sprawnego użycia:

### Temat (topic)

hierarchiczny adres wiadomości, np. `dom/garaz/ladowarka/stan`. Subskrybować można z wieloznacznikami: `+` (jeden poziom) i `#` (wszystko poniżej).

### QoS (Quality of Service)

poziom gwarancji doręczenia: 0 — „wyślij i zapomnij”, 1 — „dotrze przynajmniej raz”, 2 — „dotrze dokładnie raz”. W sieci domowej niemal zawsze wystarcza 0 lub 1.

### Retain (wiadomość zatrzymana)

broker pamięta ostatnią wiadomość z tematu i wręcza ją każdemu nowemu subskrybentowi. Dzięki temu po restarcie HA od razu zna stany, zamiast czekać na kolejne raporty.

### LWT (Last Will and Testament — „testament”)

wiadomość, którą broker opublikuje za urządzenie, gdy to zniknie bez pożegnania. Tak właśnie encje MQTT wiedzą, że urządzenie jest offline.

### Odkrywanie (MQTT Discovery)

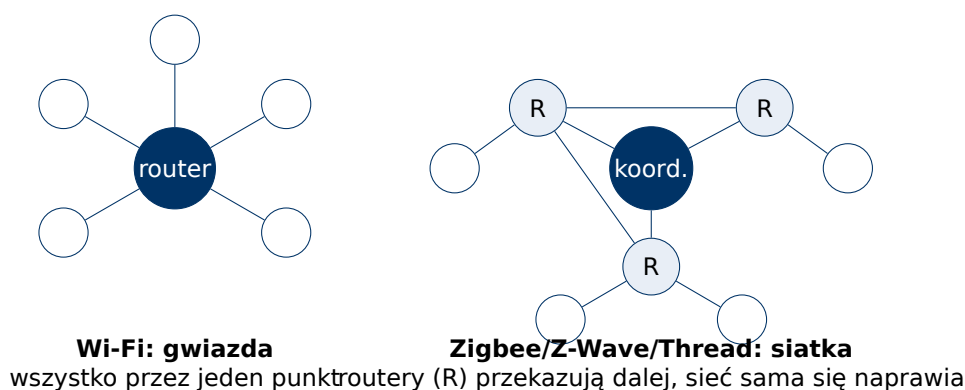
urządzenie publikuje pod specjalnym tematem `homeassistant/.../config` opis swoich encji — a HA tworzy je automatycznie, bez ręcznej konfiguracji. Z tego mechanizmu korzysta m.in. Zigbee2MQTT i Tasmota.

W praktyce: brokera (najczęściej **Mosquitto**) instaluje się jako dodatek HA OS albo osobny kontener, w HA dodaje integrację **MQTT** (adres brokera, login), a urządzenia kieruje na ten sam broker. Domyślny port to 1883 (8883 w wersji szyfrowanej TLS).

### 4.3. Zigbee — radiowa siatka czujników

**Zigbee** to radiowy standard krótkiego zasięgu (oparty o normę IEEE 802.15.4) w paśmie 2,4 GHz, zaprojektowany pod *małe zużycie energii*: czujnik drzwi potrafi żyć dwa lata na baterii pastylkowej. Ceną jest przepustowość — to sieć do meldunków „otwarto/zamknięto, 21,5 stopnia”, nie do strumieni wideo.

Sieć Zigbee jest **siatką** (mesh): urządzenia zasilane z gniazdka (żarówki, wtyczki) pełnią rolę **routerów** i przekazują pakiety dalej, a bateryjne **urządzenia końcowe** tylko meldują i śpią. Całością dyryguje **koordynator** — u nas: układ radiowy w formie modułu USB wpiętego do komputera z HA.



W HA są dwie równorzędne drogi obsługi Zigbee:

- **ZHA** (Zigbee Home Automation) — integracja wbudowana: najprostszy start, wszystko w HA;
- **Zigbee2MQTT** — osobny program, który tłumaczy Zigbee na MQTT (rozdz. 4.2); nieco więcej konfiguracji, za to największa na rynku baza obsługiwanych urządzeń i drobiazgowy raporty.

Obu naraz na jednym koordynatorze użyć nie można. Parowanie wygląda tak samo: w integracji włącza się tryb dołączania (permit join), a urządzenie wprowadza w tryb parowania (zwykle przytrzymanie przycisku); po dołączeniu następuje „przesłuchanie” (interview), w którym urządzenie opowiada, co potrafi — z tego powstają encje.

**UWAGA:** Zigbee i Wi-Fi dzielą pasmo 2,4 GHz. Koordynator wpięty bezpośrednio w port USB komputera bywa zagłuszany przez sam komputer — **zawsze używaj przedłużacza USB** (pół metra wystarczy) i trzymaj koordynator z dala od routera. Przy uporczywych problemach pomaga zmiana kanału Zigbee (15, 20 i 25 leżą między typowymi kanałami Wi-Fi 1/6/11).

### 4.4. Z-Wave — siatka na osobnej częstotliwości

**Z-Wave** działa podobnie do Zigbee (siatka, urządzenia bateryjne i routery), ale w Europie nadaje w paśmie **868 MHz**. Dłuższa fala lepiej przenika ściany i — co

równie ważne — nie tłoczy się w zatłoczonym 2,4 GHz. Urządzenia przechodzą obowiązkową certyfikację (mniejsza loteria zgodności niż w Zigbee), za co płaci się wyższą ceną. W HA obsługę zapewnia **Z-Wave JS**: osobny serwer (dodatek albo kontener) plus integracja; kontroler to znów moduł USB. Dołączanie (inclusion) nowszych urządzeń używa zabezpieczeń **S2** z kodem PIN z naklejki.

## 4.5. Thread i Matter — nowy porządek

Te dwa słowa pojawiają się dziś na pudełkach razem, ale oznaczają co innego — i to rozróżnienie warto mieć w głowie:

- **Thread** to *transport*: radiowa siatka jak Zigbee (ta sama norma IEEE 802.15.4, pasmo 2,4 GHz), tyle że każde urządzenie ma normalny adres **IPv6**. Do połączenia siatki z siecią domową potrzebny jest **border router** (router brzegowy) — tę rolę pełnią m.in. głośniki i przystawki Apple/Google oraz moduły obsługiwane przez HA.
- **Matter** to *język aplikacji*: wspólny standard opisu urządzeń („jestem żarówką, umiem ściemniać”), działający po IP — obojętnie, czy przez Thread, Wi-Fi czy Ethernet. Urządzenie łączy się kodem QR (komisjonowanie), a dzięki **multi-admin** może być jednocześnie w HA i np. w ekosystemie telefonu.

Dla nas najważniejsze: Matter jest **lokalny** (chmura niepotrzebna do działania) i wspierany w HA przez wbudowany serwer Matter. Rynek urządzeń dojrzeewa z każdym rokiem; przy zakupach nowych czujników i wtyczek warto sprawdzać logo Matter/Thread — to inwestycja w niezależność od chmur producentów.

## 4.6. Bluetooth Low Energy

**BLE** (Bluetooth Low Energy) to oszczędna odmiana Bluetooth: czujniki (termometry, wagi, opaski) okresowo *rozgłaszają* pomiary, a HA je biernie odbiera — bez parowania. Zasięg pojedynczego adaptera jest mały, ale problem elegancko rozwiązuje **Bluetooth Proxy**: dowolny moduł ESP32 (mikrokontroler za kilkanaście złotych) z oprogramowaniem ESPHome przekazuje odebrane rozgłoszenia do HA przez Wi-Fi. Kilka takich „uszu” rozstawionych po budynku daje pokrycie całości. Otwarty format rozgłoszeń nazywa się **BTHome**.

## 4.7. Kamery: RTSP i ONVIF

Świat kamer opisują dwa standardy, które znamy już z praktyki (kamera Imou w projekcie): **RTSP** (Real Time Streaming Protocol) to sposób pobierania samego strumienia wideo (adres zaczyna się od `rtsp://`, typowy port 554, kodeki H.264/H.265), a **ONVIF** — warstwa zarządzania: odkrywanie kamery, lista strumieni, sterowanie głowicą PTZ (pan-tilt-zoom), zdarzenia detekcji ruchu. W HA: integracja **ONVIF** tworzy encje kamery i zdarzeń, a podgląd w panelu serwuje wbudowany **go2rtc**, który potrafi przetłumaczyć strumień na niskolatencyjny WebRTC do przeglądarki.

**Dla ciekawych:** Nagrywanie i rozpoznawanie obiektów (osoba/samochód/zwierzę) to domena zewnętrznego programu **Frigate**, który konsumuje strumienie RTSP,

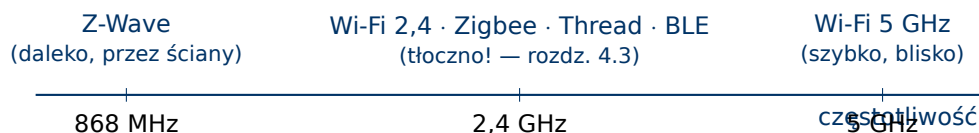
a wyniki publikuje przez MQTT — kolejny przykład, jak MQTT skleja niezależne systemy w całość.

## 4.8. Modbus — język przemysłu (i falowników)

**Modbus** pochodzi z automatyki przemysłowej lat 70. i przeżył wszystkich swoich następców dzięki prostocie: urządzenie wystawia ponumerowane **rejestry** (komórki 16-bitowe), które można czytać i zapisywać. Odmiana **RTU** biegnie po skrętce RS-485, odmiana **TCP** — po zwykłej sieci (port 502). Wartości większe niż 16 bitów składa się z dwóch rejestrów, a liczby ułamkowe przekazuje jako całkowite ze skalą (np. 2305 = 230,5 V) — dokładnie ten mechanizm znamy z symulatora falownika SUN2000 w projekcie. W HA: dedykowane integracje (np. huawei\_solar) albo uniwersalna integracja modbus, w której rejestry mapuje się ręcznie w YAML.

## 4.9. Ściąga porównawcza

| Technologia     | Pasmo         | Topologia        | Zasilanie urządzeń | Typowe zastosowanie         |
|-----------------|---------------|------------------|--------------------|-----------------------------|
| Wi-Fi           | 2,4 / 5 GHz   | gwiazda          | sieciowe           | ładowarki, kamery, gniazdka |
| Zigbee          | 2,4 GHz       | siatka           | baterie / sieciowe | czujniki, żarówki, piloty   |
| Z-Wave          | 868 MHz (EU)  | siatka           | baterie / sieciowe | zamki, rolety, czujniki     |
| Thread / Matter | 2,4 GHz / IP  | siatka / dowolna | baterie / sieciowe | nowe czujniki i wtyczki     |
| BLE             | 2,4 GHz       | rozgłoszenia     | baterie            | termometry, wagi, opaski    |
| MQTT            | (po sieci IP) | przez brokera    | —                  | spoiwo systemów             |
| Modbus          | RS-485 / IP   | żądanie-odpowieź | sieciowe           | falowniki, liczniki, HVAC   |



## 5. Ekosystemy w praktyce — czym i jak łączyć konkretne urządzenia

### 5.1. Urządzenia Tuya (w tym ładowarki AMPERE POINT)

Pełną, przeciwiczoną w boju procedurę opisują dokumenty towarzyszące (przewodnik instalacji oraz raport xtend\_tuya); tu tylko esencja architektoniczna. Urządzenie Tuya rozmawia **wyłącznie** z chmurą producenta (klasa cloud\_push przez

kanal konta aplikacji + `cloud_poll` przez OpenAPI). Wszystkie wielkości płyną jako **punkty danych** (DP, data point): ponumerowane zmienne o zadeklarowanym typie i skali. Oficjalna integracja pokazuje ich okrojony, „uprzączniony” podzbiór; dodatek **xtend\_tuya** dokłada resztę, a tryb **DP Instruction** na platformie deweloperskiej odsłania komplet. Alternatywą niezależną od chmury jest **tuya-local** (rozmowa po sieci lokalnej z użyciem klucza urządzenia) — opłacalna tam, gdzie liczy się czas reakcji i odporność na przerwy internetu.

## 5.2. ESPHome — własne urządzenia w godzinę

**ESPHome** zasługuje na osobną gwiazdkę: to projekt, w którym *opisujesz* urządzenie w YAML, a system *kompiluje* z tego gotowe oprogramowanie dla mikrokontrolera **ESP32/ESP8266**. Czujnik temperatury do rozdzielnic, licznik impulsów wodomierza, przekaźnik — wszystko sprowadza się do kilkunastu linii:

```
esphome:
  name: czujnik-rozdzielnicza

esp32:
  board: esp32dev

wifi:
  ssid: "NazwaSieci"
  password: "HasloSieci"

api:          # szyfrowane połączenie z HA (local_push)
ota:          # aktualizacje przez Wi-Fi

sensor:
  - platform: dallas_temp
    pin: GPIO4
    name: "Temperatura rozdzielniczy"
    update_interval: 30s
```

Po pierwszym wgraniu (kablem USB) kolejne aktualizacje lecą przez Wi-Fi (**OTA** — Over The Air), a HA wykrywa urządzenie sam (mDNS) i łączy się z nim natywnym, szyfrowanym API w klasie `local_push`. To najkrótsza droga od pomysłu „przydałby się czujnik X” do encji w HA — i fundament wspomnianych Bluetooth Proxy.

## 5.3. Shelly i podobne: Wi-Fi, ale lokalnie

Przekaźniki i mierniki **Shelly** pokazują, że Wi-Fi nie musi oznaczać chmury: mają lokalne API (nowsze generacje: WebSocket/RPC), integrację w HA klasy `local_push` i opcjonalnie MQTT. Montowane w puszkach za istniejącymi wyłącznikami dodają „inteligencję” bez wymiany osprzętu — a pomiar mocy w wersjach PM czyni z nich tanie liczniki częściowe do panelu Energia.

## 5.4. Telefon jako urządzenie: aplikacja Companion

Aplikacja **Home Assistant Companion** (Android/iOS) robi z telefonu pełnoprawne urządzenie: dostarcza `device_tracker` (obecność w strefach — rozdz. 9), dziesiątki sensorów (bateria, ładowanie, sieć Wi-Fi, kroki), a przede wszystkim usługę **notify** — kanał powiadomień, którego używają nasze przepisy. Powiadomienia mogą

być *aktywne* (actionable): z przyciskami, których naciśnięcie wraca do HA jako zdarzenie — przepis 9.5 pokazuje pełny obieg.

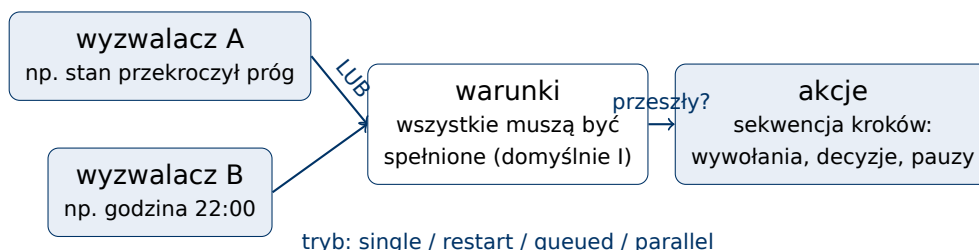
## 5.5. Co wybierać na przyszłość — krótki kompas

Do istniejącej instalacji AMPERE POINT (ładowarki Tuya + planowany falownik po Modbus) najbardziej naturalne rozszerzenia to: czujniki i wtyczki **Zigbee** (tania, dojrzała siatka — wymaga tylko koordynatora USB) lub **Matter/Thread** (nowocześniejsza alternatywa), **ESPHome** do pomiarów nietypowych oraz **Shelly** tam, gdzie trzeba wpiąć się w istniejącą elektrykę. Wspólny mianownik: wszystko działa lokalnie, więc awaria internetu zatrzymuje tylko chmurę Tuya — nie dom.

## 6. Anatomia automatyzacji

### 6.1. Trzy człony i jeden tryb

Każda automatyzacja składa się z tych samych klocków: **wyzwalaczy** (triggers — „kiedy w ogóle się obudzić?”), opcjonalnych **warunków** (conditions — „czy na pewno działać?”) i **akcji** (actions — „co zrobić?”). Nad całością czuwa **tryb** (mode), który rozstrzyga, co się stanie, gdy reguła zostanie wyzwolona ponownie, zanim skończyła poprzedni przebieg.



Dwie zasady, które oszczędzają godziny zdziwienia:

- **Wyzwalacze łączą się przez LUB** — zadziałanie któregośkolwiek budzi regułę. **Warunki łączą się przez I** — wszystkie muszą przepuścić (chyba że jawnie użyjesz or).
- Wyzwalacze reagują na *zmianę* (przekroczenie progu, wejście w stan), a nie na *trwanie*. Jeżeli moc jest powyżej progu „od zawsze”, wyzwalacz progowy nie strzeli, dopóki wartość nie spadnie i nie przekroczy progu ponownie. Do pilnowania trwania służą warunki i parametr for.

### 6.2. Nowa i stara składnia YAML — jak czytać cudze przykłady

Od wydania 2024.10 edytor zapisuje automatyzacje kluczami w liczbie mnogiej: triggers:, conditions:, actions:; typ wyzwalacza wskazuje klucz trigger:, a wywołanie usługi — action:. Starsza składnia (trigger: z platform: oraz action: z service:) **działa nadal i bezterminowo** — oba zapisy można nawet mieszać. W internecie spotkasz oba; tłumaczenie jest mechaniczne:

| Stary zapis               | Nowy zapis               |
|---------------------------|--------------------------|
| trigger: (lista)          | triggers:                |
| - platform: state         | - trigger: state         |
| condition: (lista)        | conditions:              |
| action: (lista)           | actions:                 |
| - service: switch.turn_on | - action: switch.turn_on |

### 6.3. Przegląd wyzwalaczy — z krótkimi przykładami

#### state — zmiana stanu

najczęstszy. Opcje from/to zawężają przejście, a for żąda, by nowy stan *utrzymał się* przez zadany czas (licznik zeruje się przy każdej zmianie!):

```
triggers:
- trigger: state
  entity_id: sensor.ladowarka_status
  to: "charger_charging"
  for: "00:00:10"
```

#### numeric\_state — próg liczbowy

strzela w momencie *przekroczenia* progu (above/below), również z for:

```
triggers:
- trigger: numeric_state
  entity_id: sensor.moc_eksportu
  above: 3000
  for: "00:05:00"
```

#### time i time\_pattern

konkretna godzina (at: "22:00:00", też encja input\_datetime) albo rytm (time\_pattern z minutes: "/5" = co pięć minut). Rytmów używaj oszczędnie — to odpytywanie świata na siłę; zwykle lepszy jest wyzwalacz zdarzeniowy.

#### sun — słońce

wschód/zachód z przesunięciem: event: sunset, offset: "-00:30:00". Do progów jasności lepsza bywa elewacja słońca w warunku szablonowym.

#### template — dowolny warunek jako wyzwalacz

strzela, gdy szablon (rozdz. 7) zmieni wynik na prawdę; z for pilnuje trwania. Najpotężniejszy i najłatwiejszy do przekombinowania.

#### zone — strefy

wejście/wyjście osoby (śledzonej przez aplikację Companion) ze strefy zdefiniowanej na mapie: zone: zone.dom, event: enter.

#### webhook

unikalny adres URL; żądanie HTTP pod ten adres wyzwala regułę — najprostszy most z zewnętrznych systemów.

#### event, mqtt, device

surowe zdarzenie z magistrali (np. akcja z powiadomienia — przepis 9.5), wiadomość MQTT na temacie, albo „wyzwalacz urządzenia” generowany przez edytor. Ten ostatni jest wygodny w kreatorze, ale w ręcznym YAML czytelniejsze są wyzwalacze encji.

**homeassistant — start systemu**

event: start pozwala odtworzyć pożądane stany po restarcie (patrz pułapki, rozdz. 10).

Każdemu wyzwalaczowi można nadać id — etykietę, po której akcje rozpoznają, co obudziło regułę (używa jej przepis taryfowy w rozdz. 9).

**6.4. Warunki**

Warunki mają te same rodzaje co wyzwalacze (state, numeric\_state, time, sun, zone, template) plus spójniki and/or/not. Dwa niuanse:

- time jako warunek przyjmuje też dni tygodnia: weekday: [mon, tue, wed, thu, fri].
- Warunek state z for pyta „czy encja jest w tym stanie *co najmniej od...*” — to uzupełnienie wyzwalacza, nie duplikat.

**6.5. Akcje — pełny repertuar**

Akcje wykonują się po kolei, jak skrypt. Poza zwykłymi wywołaniami (action: switch.turn\_on z target:) masz do dyspozycji strukturę sterującą:

**choose / default**

rozgałęzienie „pierwszy pasujący wariant”: lista opcji, każda z własnymi warunkami i sekwencją. Idealne w parze z trigger\_id.

**if / then / else**

prostsza, dwugałęziowa wersja choose.

**repeat**

pętla: count (n razy), while (dopóki warunek), until (aż warunek). Wewnątrz dostępny licznik repeat.index.

**wait\_for\_trigger / wait\_template**

zatrzymaj się i czekaj na zdarzenie albo na spełnienie szablonu; zawsze dodawaj timeout i decyduj o continue\_on\_timeout, inaczej reguła może wisieć godzinami.

**delay**

pauza. Pamiętaj: restart HA przerywa trwające delay i wait — automatyzacja „zapomina”, że czekała. Długie odliczania bezpieczniej realizować pomocnikiem timer, który potrafi wznowić się po restarcie.

**parallel**

gałęzie równoległe (np. powiadom trzy osoby naraz).

**stop**

zakończ przebieg (z opcjonalnym komunikatem w śladzie); wariant z error: t true oznacza przebieg jako błędny.

**variables**

zmienne lokalne przebiegu — policz raz, użyj wielokrotnie (przepis PV, rozdz. 9.3).

## 6.6. Tryby wykonania

| Tryb              | Zachowanie przy ponownym wyzwoleniu w trakcie pracy                                                            |
|-------------------|----------------------------------------------------------------------------------------------------------------|
| single (domyślny) | nowy przebieg jest <i>pomijany</i> (w dzienniku: ostrzeżenie). Dobry do reguł, które nie powinny się nakładać. |
| restart           | bieżący przebieg jest przerywany, rusza nowy. Naturalny dla „światło na 5 minut od ostatniego ruchu”.          |
| queued            | przebiegi ustawiają się w kolejkę (limit max). Dla sekwencji, które muszą wykonać się wszystkie, po kolei.     |
| parallel          | przebiegi biegą równolegle (limit max). Dla reguł obsługujących wiele niezależnych obiektów naraz.             |

**UWAGA:** Najczęstsza zagadka „czemu automatyzacja nie zadziałała” u początkujących to tryb single + długie delay/wait w akcjach: reguła wciąż „pracuje”, więc kolejne wyzwolenia przepadają. Rozwiązania: tryb restart, krótsze czekania albo przeniesienie odliczania do pomocnika timer.

## 7. Szablony Jinja2 — język, w którym HA myśli

**Jinja2** to język szablonów: tekst, w którym fragmenty w podwójnych klamrach `{{ ... }}` są obliczane, a bloki `{% ... %}` sterują logiką. Szablony pojawiają się wszędzie: w wyzwalaczach template, warunkach, treściach powiadomień, encjach szablonowych. Poligonem doświadczalnym jest **Narzędzia deweloperskie** → **Szablon** — wynik widać na żywo, w miarę pisanía.

### 7.1. Absolutna podstawa: stany są tekstem

Funkcja `states('sensor.moc')` zwraca stan encji — **zawsze jako tekst**, nawet gdy wygląda jak liczba. Porównanie tekstu z liczbą to klasyczna pułapka: `'7' > 30` nie znaczy tego, co podpowiada intuicja. Dlatego liczby zawsze konwertujemy filtrem, z wartością awaryjną na wypadek stanu nieliczbowego:

```
{{ states('sensor.moc_eksportu') | float(0) }}
{{ (states('sensor.energia') | float(0)) > 3.5 }}
```

Wartość awaryjna (tu: 0) ratuje przed stanami unknown (jeszcze nieznany) i unavailable (integracja straciła urządzenie) — oba zdarzają się każdemu czujnikowi, choćby po restarcie. Czystszy test dostępności: `has_value('sensor.moc')`.

## 7.2. Niezbędnik — funkcje i filtry, które załatwiają 95% potrzeb

| Wyrażenie                                                                                                                           | Co robi                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>states('sensor.x')</code>                                                                                                     | stan encji jako tekst                                                                                                                                 |
| <code>is_state('switch.x', 'on')</code>                                                                                             | czy encja jest w danym stanie (bezpieczne porównanie)                                                                                                 |
| <code>state_attr(enc, 'atrybut')</code>                                                                                             | wartość atrybutu encji (albo None), np. ('sun.sun', 'elevation')                                                                                      |
| <code>has_value('sensor.x')</code><br><code>  float(0),   int(0)</code><br><code>  round(1)</code><br><code>  default('---')</code> | czy stan jest liczbowo/logicznie użyteczny<br>konwersja na liczbę z wartością awaryjną<br>zaokrąglenie<br>wartość zastępcza, gdy zmienna nie istnieje |
| <code>now()</code>                                                                                                                  | bieżący czas (obiekt datetime, strefa lokalna)                                                                                                        |
| <code>today_at('06:30')</code>                                                                                                      | dzisiejsza data o podanej godzinie                                                                                                                    |
| <code>as_timestamp(x)</code>                                                                                                        | czas na sekundy epoki (do arytmetyki)                                                                                                                 |
| <code>timedelta(minutes=15)</code>                                                                                                  | odcinek czasu do dodawania/odejmowania                                                                                                                |
| <code>states.sensor.x.last_changed</code>                                                                                           | kiedy stan ostatnio się zmienił                                                                                                                       |
| <code>expand('group.czujniki')</code><br><code>  selectattr(...)   list</code>                                                      | rozwińnięcie grupy do listy encji<br>filtrowanie list (np. tylko encje w stanie on)                                                                   |

Dwa przykłady „z życia”, które łączą klocki:

```
{# Ile minut trwa bieżąca sesja ładowania? #}
{{ ((now() - states.sensor.ladowarka_status.last_changed).total_seconds() / 60) |
  round(0) }}

{# Ktore czujniki sa niedostepne? #}
{{ states.sensor | selectattr('state','in',['unknown','unavailable'])
  | map(attribute='entity_id') | list | join(', ') }}
```

**Wskazówka:** W wyzwalaczu i warunku template obowiązuje subtelna zasada: wyzwalacz przelicza się tylko wtedy, gdy zmieni się któraś z encji *użytych w szablonie* (HA sam je wykrywa). Szablon bez żadnej encji — np. sam `now()` — nie będzie się odświeżał tak, jak intuicja podpowiada; do reguł czasowych służą wyzwalacze `time/time_pattern`.

## 8. Elementy pomocnicze: helpery, skrypty, sceny, blueprinty

### 8.1. Helpery — pamięć i pokrętła Twoich reguł

Helpery (**Ustawienia** → **Urządzenia i usługi** → **Pomocnicy**) to encje tworzone ręcznie, bez fizycznego urządzenia. To one czynią automatyzacje konfigurowalnymi i odpornymi na restart:

#### **input\_boolean**

wirtualny przełącznik — flaga trybu („goście”, „wakacje”, „serwis”). Automatyzacje sprawdzają go w warunkach; Ty przełączasz z pulpitu.

**input\_number**

liczba z suwakiem — progi i nastawy (cena prądu, minimalna nadwyżka PV). Zmieniasz próg bez edycji reguły.

**input\_select**

lista opcji — tryby pracy („Eko / Komfort / Poza domem”).

**input\_datetime, input\_text**

godzina/data i tekst do wpisania.

**timer**

odliczanie z encją i zdarzeniem `timer.finished`; w przeciwieństwie do `delay` ma opcję przetrwania restartu (`restore`).

**counter**

licznik zdarzeń (ile sesji ładowania w tym tygodniu?).

**schedule**

tygodniowy grafik „włącz/wyłącz” rysowany myszką — encja jest `on` w zaznaczonych oknach. Wymarzony do taryf (przepis 9.1).

**grupa, próg, pochodna, statystyki, Riemann**

gotowe encje przetwarzające: grupowanie, binarne przekroczenie progu, pochodna (szybkość zmian), statystyki kroczące oraz całka Riemanna — najważniejsza z nich dla energetyki (rozdz. 11).

## 8.2. Skrypty i sceny

**Skrypt** to nazwana sekwencja akcji — wielokrotnego użytku, wywoływana z automatyzacji, pulpitu albo głosowo. Skrypt może przyjmować parametry (sekcja `fields`) i — od nowszych wydań — zwracać wynik (`response_variable`). Reguła kciuka: logika, którą chcesz wywołać z więcej niż jednego miejsca, idzie do skryptu; automatyzacje tylko decydują *kiedy*.

**Scena** to zapamiętany zestaw stanów (światła 30%, rolety w dół). `scene.turn_on` przywraca zestaw; sprytny wariant: `scene.create` robi „migawkę” bieżących stanów, a po chwili można do niej wrócić — np. mrugnij światłami na czerwono przy błędzie ładowarki, potem odtwórz, co było.

## 8.3. Blueprinty — automatyzacje z formularzem

**Blueprint** to szablon automatyzacji z polami do wypełnienia („wybierz czujnik ruchu”, „wybierz światło”). Z jednego blueprintu tworzy się dziesiątki instancji; społeczność publikuje setki gotowych (import: **Ustawienia** → **Automatyzacje i sceny** → **Blueprinty** → **Importuj**, wklejenie adresu URL). Dla firmy takiej jak AMPERE POINT blueprint to także sposób dystrybucji sprawdzonych reguł do klientów — jedna dopracowana „ładowarka po taniej taryfie” zamiast indywidualnego YAML u każdego.

## 9. Przepisy — kompletne automatyzacje z komentarzem

Wszystkie przepisy używają przykładowych identyfikatorów (`switch.ładowarka`, `sensor.ładowarka_status` itd.) — podmień je na własne z zakładki Encje.

Statusy ładowarek serii Q przyjmują wartości `charger_free`, `charger_insert`, `charger_wait`, `charger_charging`, `charger_pause`, `charger_end`, `charger_fault` (Q37: encja `x_work_state` o analogicznych stanach). Każdy przepis wklejysz w edytorze automatyzacji przez menu „...” → **Edytuj w YAML**.

### 9.1. Ładowanie w taniej taryfie — wersja dojrzała (helper schedule)

Zamiast wpisywać godziny w regułę, rysujemy grafik: pomocnik **schedule** o nazwie `schedule.tania_taryfa` (np. 22-6 w dni robocze, całe weekendy). Automatyzacja robi się trywialna, a zmiana taryfy to edycja grafiku, nie kodu:

```
alias: Ładowarka wg taniej taryfy
description: Włącza ładowanie w oknach grafiku, wyłącza poza nimi
mode: single
triggers:
  - trigger: state
    entity_id: schedule.tania_taryfa
    id: zmiana
actions:
  - choose:
    - conditions:
      - condition: state
        entity_id: schedule.tania_taryfa
        state: "on"
      sequence:
        - action: switch.turn_on
          target: { entity_id: switch.ladowarka }
    default:
      - action: switch.turn_off
        target: { entity_id: switch.ladowarka }
```

Komentarz: wyzwaczem jest *każda* zmiana stanu grafiku (wejście i wyjście z okna); `choose` z gałęzią domyślną załatwia obie strony jedną regułą. Chcesz wyjątek „ładuj natychmiast”? Dodaj `input_boolean.laduj_teraz` i warunek `off` na nim w gałęzi wyłączającej.

### 9.2. Powiadomienie o końcu ładowania — z danymi sesji

```
alias: Koniec ładowania - powiadomienie
mode: single
triggers:
  - trigger: state
    entity_id: sensor.ladowarka_status
    from: "charger_charging"
    to:
      - "charger_end"
      - "charger_free"
actions:
  - action: notify.mobile_app_telefon
    data:
      title: "Ładowarka"
      message: >-
        Ładowanie zakończone. Sesja:
        {{ states('sensor.ladowarka_energia_sesji') | float(0) | round(2) }} kWh
        ({{ (states('sensor.ladowarka_energia_sesji') | float(0)
          * states('input_number.cena_kwh') | float(1)) | round(2) }} zł).
```

Komentarz: to: przyjmuje listę — łapiemy i naturalny koniec (charger\_end), i wypięcie wtyczki. Koszt liczy szablon z pomocnika input\_number.cena\_kwh — cena prądu jest nastawą na pulpicie, nie liczbą zaszytą w regule.

### 9.3. Flagowy: ładowanie nadwyżką z fotowoltaiki z płynną regulacją prądu

Cel: gdy dom oddaje energię do sieci, ładowarka ma ją przechwycić — i *dopasowywać prąd* do bieżącej nadwyżki, zamiast tylko włączać się i wyłączać. Założenia: czujnik sensor.moc\_eksportu (dodatni = oddajemy; z falownika po Modbus albo z naszego symulatora SUN2000), ładowarka trójfazowa z nastawą number.ladowarka\_limit\_pradu (6-16 A), histereza w pomocnikach: input\_number.pv\_prog\_startu (np. 4200 W) i input\_number.pv\_prog\_stopu (np. 800 W).

```
alias: Ładowanie nadwyżka PV (3 fazy, płynna regulacja)
description: >-
  Start powyżej progu startu, płynne dopasowanie prądu co minute,
  stop po spadku poniżej progu stopu utrzymanym 5 minut.
mode: single
triggers:
  - trigger: numeric_state
    entity_id: sensor.moc_eksportu
    above: input_number.pv_prog_startu
    for: "00:03:00"
    id: start
  - trigger: time_pattern
    minutes: "/1"
    id: koryguj
  - trigger: numeric_state
    entity_id: sensor.moc_eksportu
    below: input_number.pv_prog_stopu
    for: "00:05:00"
    id: stop
conditions:
  - condition: state
    entity_id: sensor.ladowarka_status
    state:
      - "charger_insert"
      - "charger_wait"
      - "charger_charging"
      - "charger_pause"
actions:
  - variables:
      eksport: "{{ states('sensor.moc_eksportu') | float(0) }}"
      pobor_ladowarki: "{{ states('sensor.ladowarka_moc') | float(0) }}"
      do_dyspozycji: "{{ eksport + pobor_ladowarki }}"
      prad: >-
        {{ [16, [6, (do_dyspozycji / (3 * 230)) | int] | max] | min }}
  - choose:
      - conditions: "{{ trigger.id == 'stop' }}"
        sequence:
          - action: switch.turn_off
            target: { entity_id: switch.ladowarka }
      - conditions: "{{ trigger.id == 'start' }}"
        sequence:
          - action: number.set_value
            target: { entity_id: number.ladowarka_limit_pradu }
            data: { value: "{{ prad }}" }
          - action: switch.turn_on
```

```

    target: { entity_id: switch.ladowarka }
  - conditions: >-
    {{ trigger.id == 'koryguj'
      and is_state('switch.ladowarka','on')
      and (prad - states('number.ladowarka_limit_pradu')|float(0)) | abs >= 1
    }}
  sequence:
  - action: number.set_value
    target: { entity_id: number.ladowarka_limit_pradu }
    data: { value: "{{ prad }}" }

```

Komentarz linia po linii, bo to najgęstszy przepis poradnika:

- **Trzy wyzwalacze z id:** start (nadwyżka utrzymana 3 min — chmura przelatująca nad domem nie wystartuje ładowania), korekta co minutę, stop (spadek utrzymany 5 min — auto nie „mruga”). Progi są encjami `input_number`, więc stroisz je z pulpitu.
- **Warunek wspólny:** pojazd w ogóle podłączony (lista akceptowanych stanów).
- **Kluczowa zmienna do dyspozycji:** do bieżącego eksportu *dodajemy* aktualny pobór ładowarki — bo gdy ładowarka już pracuje, jej pobór pomniejszył eksport; bez tej poprawki reguła „dusiłaby” sama siebie (najczęstszy błąd w tego typu automatyzacjach w internecie!).
- **Przeliczenie na prąd:** moc dzielona przez  $3 \times 230 \text{ V}$  (trzy fazy; dla ładowarki jednofazowej usuń „3 \*”), obciąża do widełek 6–16 A zapisem „min-max” na listach Jinja2.
- **Korekta tylko przy różnicy  $\geq 1 \text{ A}$**  — oszczędzamy urządzeniu ciągłego przestawiania o ułamki ampera.
- Tryb single: gdyby przebieg jeszcze trwał, kolejny tik minutowy po prostu przepadnie — tu to pożądane.

**UWAGA:** Minimalny prąd ładowania według normy IEC 61851 to 6 A — poniżej auto przerywa sesję. Dlatego widełki zaczynają się od 6 A, a start ma osobny, wyższy próg mocy:  $6 \text{ A} \times 3 \times 230 \text{ V} \approx 4,1 \text{ kW}$  nadwyżki musi być, żeby w ogóle ruszyć trójfazowo.

## 9.4. Alarm błędu z eskalacją — aż ktoś potwierdzi

Pomocnik: `input_boolean.alarm_potwierdzony`. Reguła powiadamia co 10 minut, maksymalnie 6 razy, dopóki ktoś nie przełączy potwierdzenia (z pulpitu albo przyciskiem w powiadomieniu — patrz następny przepis):

```

alias: Bład ładowarki - eskalacja
mode: single
triggers:
  - trigger: state
    entity_id: sensor.ladowarka_status
    to: "charger_fault"
actions:
  - action: input_boolean.turn_off
    target: { entity_id: input_boolean.alarm_potwierdzony }
  - repeat:
    until:
      - condition: or
        conditions:
          - condition: state

```

```

        entity_id: input_boolean.alarm_potwierdzony
        state: "on"
      - "{{ repeat.index >= 6 }}"
    sequence:
      - action: notify.mobile_app_telefon
        data:
          title: "UWAGA: blad ladowarki ({{ repeat.index }}/6)"
          message: "Stan: {{ states('sensor.ladowarka_status') }}"
      - wait_for_trigger:
          - trigger: state
            entity_id: input_boolean.alarm_potwierdzony
            to: "on"
        timeout: "00:10:00"
        continue_on_timeout: true

```

Komentarz: `repeat.until` sprawdza warunek *po* każdym obiegu; `wait_for_trigger` z `timeout` realizuje „czekaj 10 minut albo do potwierdzenia — co pierwsze”. Zwróć uwagę na zerowanie flagi na początku: każdy nowy błąd zaczyna eskalację od czysta.

## 9.5. Powiadomienie z przyciskami (actionable) — pełny obieg

Dwie reguły. Pierwsza wysyła powiadomienie z przyciskiem; druga nasłuchuje zdarzenia, które aplikacja Companion odsyła po naciśnięciu:

```

alias: Auto podlaczone - zapytaj o start
mode: single
triggers:
  - trigger: state
    entity_id: sensor.ladowarka_status
    to: "charger_insert"
actions:
  - action: notify.mobile_app_telefon
    data:
      title: "Ladowarka"
      message: "Auto podlaczone. Rozpoczac ladowanie?"
      data:
        actions:
          - action: "START_LADOWANIA"
            title: "Laduj teraz"
          - action: "POTWIERDZ_ALARM"
            title: "Potwierdz alarm"

```

```

alias: Obsługa przyciskow z powiadomien
mode: parallel
triggers:
  - trigger: event
    event_type: mobile_app_notification_action
    event_data: { action: "START_LADOWANIA" }
    id: laduj
  - trigger: event
    event_type: mobile_app_notification_action
    event_data: { action: "POTWIERDZ_ALARM" }
    id: potwierdz
actions:
  - choose:
      - conditions: "{{ trigger.id == 'laduj' }}"
        sequence:
          - action: switch.turn_on
            target: { entity_id: switch.ladowarka }
      - conditions: "{{ trigger.id == 'potwierdz' }}"

```

```
sequence:
  - action: input_boolean.turn_on
    target: { entity_id: input_boolean.alarm_potwierdzony }
```

Komentarz: przycisk nie wywołuje niczego bezpośrednio — odsyła *zdarzenie* na magistralę (rozdz. 2), a dopiero reguła-odbiornik decyduje, co ono znaczy. Tryb *parallel*: naciśnięcia z dwóch telefonów nie będą się blokować.

## 9.6. Watchdog niedostępności — system pilnuje sam siebie

```
alias: Watchdog - encje ładowarki niedostępne
mode: single
triggers:
  - trigger: state
    entity_id:
      - sensor.ladowarka_status
      - sensor.ladowarka_moc
    to:
      - "unavailable"
      - "unknown"
    for: "00:10:00"
actions:
  - action: notify.mobile_app_telefon
    data:
      title: "HA: utracono kontakt z ładowarką"
      message: >-
        {{ trigger.entity_id }} jest {{ trigger.to_state.state }}
        od 10 minut. Sprawdź internet / zasilanie / chmure Tuya.
```

Komentarz: *for* odsiewa sekundowe czknięcia chmury; zmienna *trigger* w szablonie mówi, *która* encja i w *jakim* stanie — jedna reguła obsługuje dziesiątki encji.

## 9.7. Raport dzienny o 21:00

Zakłada licznik dzienny `sensor.ladowarka_energia_dzienna` z pomocnika `utility_meter` (rozdz. 11):

```
alias: Raport dzienny ładowania
mode: single
triggers:
  - trigger: time
    at: "21:00:00"
conditions:
  - condition: numeric_state
    entity_id: sensor.ladowarka_energia_dzienna
    above: 0.1
actions:
  - action: notify.mobile_app_telefon
    data:
      title: "Raport ładowania - {{ now().strftime('%d.%m') }}"
      message: >-
        Dzis naładowano
        {{ states('sensor.ladowarka_energia_dzienna') | float(0) | round(1) }} kWh
        za ok. {{ (states('sensor.ladowarka_energia_dzienna') | float(0)
          * states('input_number.cena_kwh') | float(1)) | round(2) }} zł.
```

Komentarz: warunek „powyżej 0,1 kWh” wycisza raport w dni bez ładowania; format daty ustawia `strftime` — ten sam mechanizm przyda się w każdym powiadomieniu z datą.

## 10. Diagnostyka — gdy reguła nie robi tego, co miała

### 10.1. Ślady (traces) — najpotężniejsze i najmniej znane narzędzie

Każdy przebieg automatyzacji zostawia **ślad**: edytor reguły → zakładka **Ślady**. Zobaczysz graf wykonania krok po kroku: który wyzwalacz strzelił, jak wypadły warunki, którą gałąź choose wybrano, jakie wartości miały zmienne — łącznie z pełnym obiektem trigger. HA trzyma domyślnie pięć ostatnich śladów na regułę. Dziewięć na dziesięć zagadek rozwiązuje się właśnie tutaj, bez zgadywania.

### 10.2. Reszta niezbędnika

- **Dziennik** (Logbook) — oś czasu „co się działo”: zmiany stanów i kto/co je wywołało.
- **Narzędzia deweloperskie** → **Stany** — podgląd i *ręczne ustawienie* dowolnego stanu (tylko do testów — integracja nadpisze przy najbliższym raporcie). Nieocenione do prowokowania reguł.
- **Narzędzia deweloperskie** → **Zdarzenia** — nasłuch magistrali (np. `mobile_app_notification_action` przy debugowaniu przycisków powiadomień).
- **Narzędzia deweloperskie** → **Szablon** — poligon Jinja2 na żywo.
- **Logi** (Ustawienia → System → Logi) z filtrem po nazwie integracji; poziom szczegółowości podnosi się usługą `logger.set_level` bez restartu.

### 10.3. Pułapki, które łapią każdego

- **Tryb single + długie czekanie** — kolejne wyzwolenia przepadają (rozd. 6).
- **for zeruje się przy każdej zmianie stanu** — czujnik migoczący wokół progu nigdy nie „uzbiera” czasu; pomaga histereza (dwa progi) albo pomocnik *próg* z wbudowaną histerezą.
- **Wyzwalacz progowy nie strzela „bo już jest powyżej”** — `numeric_state` reaguje na *przekroczenie*, nie na trwanie. Po restarcie HA nic nie „przekracza” — stany dopiero się ładują. Reguły krytyczne uzupełnij wyzwalaczem `homeassistant: start` plus warunkami.
- **unknown/unavailable w arytmetyce** — zawsze `| float(0) / has_value()` (rozd. 7); jedna niedostępna encja potrafi po cichu wyłączyć całą regułę na dobę.
- **Restart przerywa delay/wait** — długie odliczania przenoś do pomocnika `timer` z opcją `restore`.
- **Pętla sprzężenia** — reguła zmienia encję, którą sama jest wyzwalana. Ślady pokażą to od razu; rozwiązaniem bywa warunek odcinający albo rozdzielenie na dwie reguły.

## 11. Pomiary energii — od encji do panelu Energia

### 11.1. Co encja musi mieć, żeby panel ją przyjął

Panel Energia przyjmuje wyłącznie encje-liczniki: `device_class: energy`, jednostka kWh oraz `state_class: total_increasing` — czyli wartość *narastającą* (jak licznik w rozdzielnicy), która może się co najwyżej wyzerować (HA rozpoznaje zerowanie i policzy przyrost poprawnie). Chwilowa moc (W) ma `state_class: measurement` i do panelu jako źródło energii się nie nadaje — to najczęstsze nieporozumienie.

W ładowarkach serii Q rolę licznika pełni `forward_energy_total`. Gdy urządzenie daje tylko moc, energię wytwarza pomocnik **Integracja — suma Riemanna** (całkuje moc po czasie; wybierz metodę *lewostronną* — dla skokowych przebiegów mocy nie zawyża wyniku).

### 11.2. Taryfy i okresy: `utility_meter`

Pomocnik **`utility_meter`** to licznik źródłowy na okresy (dzienny, miesięczny — stąd `sensor.ladowarka_energia_dzienna` w przepisie raportu) i opcjonalnie na **taryfy**. Przy taryfach pomocnik tworzy encję wyboru; przełączanie taryfy to jedna automatyzacja sprzężona z grafiką z przepisu 9.1:

```
triggers:
- trigger: state
  entity_id: schedule.tania_taryfa
actions:
- action: select.select_option
  target: { entity_id: select.energia_ladowarki }
  data:
    option: >-
      {{ 'tania' if is_state('schedule.tania_taryfa','on') else 'droga' }}
```

Koszty w panelu Energia: cena stała, encja ceny (np. z pomocnika `input_number.cena_kwh`) albo cennik godzinowy z integracji giełdowej — panel przeliczy zużycie na złotówki za każdy okres.

## 12. Niezawodność, kopie zapasowe, dostęp zdalny

### 12.1. Kopie zapasowe

W HA OS: **Ustawienia** → **System** → **Kopie zapasowe** — od wydań 2025 z harmonogramem automatycznym i szyfrowaniem; kopię *pobieraj* poza urządzenie (karta SD, która umarła, zabiera też kopię na siebie). W wariantcie kontenerowym kopią jest folder `config` (nasz przewodnik, rozdz. o kopiach). Rytuał minimum: kopia przed każdą aktualizacją i raz w tygodniu automatycznie.

### 12.2. Aktualizacje bez dramatów

Czytaj noty wydania (sekcja *Breaking changes* — zmiany niekompatybilne); nie aktualizuj w dniu premiery, chyba że lubisz przygody — poprawkowe wydanie .1-.3 wychodzi zwykle w ciągu dni; po aktualizacji przejrzyj **Ustawienia** → **System**

→ **Naprawy** i logi. Dodatki społecznościowe (xtend\_tuya!) sprawdzaj pod kątem zgodności *przed* dużym skokiem wersji rdzenia.

### 12.3. Dostęp spoza domu — trzy drogi

#### Home Assistant Cloud (Nabu Casa)

abonament: szyfrowany tunel bez konfiguracji, plus asystenci głosowi; przy okazji finansuje rozwój projektu. Najprostsze i bezpieczne.

#### VPN (Virtual Private Network — wirtualna sieć prywatna)

np. WireGuard albo Tailscale: telefon „wchodzi” do sieci domowej i widzi HA jak z kanapy. Darmowe, bardzo bezpieczne, wymaga kwadransa konfiguracji.

#### Wystawienie portu / reverse proxy

dla zaawansowanych: własna domena, certyfikaty TLS, obowiązkowo uwierzytelnianie dwuskładnikowe. Największa kontrola — i największa odpowiedzialność; nigdy nie wystawiaj HA „gołym” portem 8123 bez HTTPS.

Niezależnie od drogi: załóż osobne konta domownikom (bez uprawnień administratora), włącz uwierzytelnianie dwuskładnikowe na kontach administracyjnych, a do integracji zewnętrznych używaj *tokenów długoterminowych* zamiast hasła.

## 13. Najczęstsze problemy — ściągą

| Objaw                                             | Pierwszy ruch                                                                                                                             |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Automatyzacja „czasem nie działa”                 | Zakładka Ślady: który wyzwalacz, które warunki. W 90% winne: tryb single albo warunek na encji unavailable.                               |
| Reguła progowa nie startuje po restarcie          | numeric_state reaguje tylko na przekroczenie; dodaj wyzwalacz startu HA + warunki.                                                        |
| Powiadomienia nie przychodzą                      | Sprawdź nazwę usługi notify.mobile_app_* (Narzędzia deweloperskie → Akcje), zezwolenia aplikacji na telefonie, tryb oszczędzania baterii. |
| Przycisk z powiadomienia nic nie robi             | Nasłuchaj zdarzenia mobile_app_notification_action w Narzędziach deweloperskich → Zdarzenia — literówka w action to klasyk.               |
| Panel Energia nie widzi encji                     | Brak state_class: total_increasing / złej jednostki; użyj licznika (forward_energy_total), nie mocy.                                      |
| Wartości energii „skaczą” po restarcie urządzenia | Licznik urządzenia się wyzerował — total_increasing to obsłuży; nie używaj total bez potrzeby.                                            |
| Zigbee gubi urządzenia                            | Koordinator na przedłużaczu USB, z dala od routera i dysków USB 3.0; dołóż routery (wtyczki); rozważ kanał 15/20/25.                      |

| Objaw                                   | Pierwszy ruch                                                                                                                |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Encje chmurowe co noc unavailable       | Restart routera / przerwa u operatora / serwis chmury — watchdog z przepisów odsieje sekundowe czknięcia od realnych awarii. |
| Szablon działa w testerze, w regule nie | W testerze patrzysz na wynik, w wyzwalaczu liczy się <i>zmiana</i> wyniku i encje wykryte w szablonie; sprawdź ślad.         |
| Po aktualizacji HA coś zniknęło         | Ustawienia → System → Naprawy + noty wydania; dodatki HACS mogły wymagać własnej aktualizacji.                               |

## 14. Słowniczek pojęć i skrótów

### Akcja (action)

pojedynczy krok automatyzacji: wywołanie usługi, decyzja, pauza.

**API** interfejs programistyczny — „okienko podawcze” dla programów.

### BLE / BTHome

energooszczędny Bluetooth / otwarty format rozgłoszeń czujników.

### Blueprint

szablon automatyzacji z polami do wypełnienia.

### Border router

urządzenie łączące siatkę Thread z siecią domową IP.

### Broker

pośrednik MQTT rozdzielający wiadomości (np. Mosquitto).

### Config entry (wpis konfiguracyjny)

zapisany wynik kreatora integracji; bez niego integracja nie działa.

### DP (data point)

ponumerowana zmienna urządzenia Tuya (typ + skala).

### Encja / urządzenie / obszar

wartość → teczka wartości → pomieszczenie.

### ESPHome

system budowy własnych urządzeń na ESP32 z konfiguracji YAML.

### Helper (pomocnik)

encja tworzona ręcznie: flaga, liczba, grafik, licznik, minutnik.

### HTTP / REST / WebSocket

protokół stron www / styl API „pobierz-ustaw” / stałe łącze dwukierunkowe.

### IoT class

deklaracja integracji: lokalnie czy chmurowo, push czy poll.

### Jinja2

język szablonów HA ({{ ... }}).

### LWT

„testament” MQTT — wiadomość publikowana przez brokera po zniknięciu urządzenia.

**Magistrala zdarzeń (event bus)**

wewnętrzna tablica ogłoszeń HA; wszystko jest zdarzeniem.

**Matter / Thread**

wspólny język urządzeń po IP / radiowa siatka IPv6 (IEEE 802.15.4).

**mDNS / SSDP / DHCP**

mechanizmy, którymi HA wykrywa urządzenia w sieci.

**Mesh (siatka)**

topologia, w której urządzenia przekazują pakiety dalej (Zigbee, Z-Wave, Thread).

**Modbus (RTU/TCP)**

przemysłowy protokół rejestrów 16-bitowych (RS-485 / port 502).

**MQTT / QoS / retain**

protokół publikuj-subskrybuj / poziom gwarancji doręczenia / wiadomość zatrzymana.

**ONVIF / RTSP**

standard zarządzania kamerami / protokół strumienia wideo.

**OTA**

aktualizacja urządzenia „przez powietrze”.

**Recorder / SQLite**

komponent zapisu historii / domyślna baza danych w jednym pliku.

**Scena / skrypt**

zapamiętany zestaw stanów / nazwana sekwencja akcji.

**state\_class (measurement / total\_increasing)**

klasa statystyk encji: wartość chwilowa / licznik narastający.

**Ślad (trace)**

zapis przebiegu automatyzacji krok po kroku.

**Tryb (mode)**

zachowanie reguły przy ponownym wyzwoleniu: single, restart, queued, parallel.

**utility\_meter**

pomocnik tnący licznik na okresy i taryfy.

**Warunek / wyzwalacz**

filtr „czy działać” / zdarzenie budzące regułę.

**Webhook**

adres URL wyzwalający automatyzację żądaniem HTTP.

**ZHA / Zigbee2MQTT**

dwie drogi obsługi Zigbee w HA (wbudowana / przez MQTT).

**Z-Wave JS**

serwer i integracja obsługi Z-Wave.

---

Koniec dokumentu — wersja 1, 2026-07-06. Kolejne wersje oznaczać v2, v3... bez usuwania poprzednich.

Dokumenty powiązane: przewodnik instalacji (v2), raport xtend\_tuya (v1), poradnik kliencki www (v2), pv\_sim (symulator falownika).